

LP3 (ISR) – Assignment 3 – Inverted Files (RollNo: 43221)

Input Code:

```
import java.io.*;
import java.util.*;

public class InvertedFile {
    public static void main(String[] args) throws NumberFormatException,
        IOException {
        String fname = "";
        ArrayList<String> invertedData = new ArrayList<String>();
        int docno[][] = new int[100][10];
        InputStreamReader ins = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(ins);
        System.out.println("\nEnter TOTAL NO OF FILES:");
        int no = Integer.parseInt(br.readLine());
        int i = 1;
        while (i - 1 != no) {
            System.out.println("\nEnter FILE " + i + " NAME:");
            fname = br.readLine();
            indexing(fname, invertedData, docno, i);
            i += 1;
        }
        displayIndex(invertedData, docno);
    }

    public static void displayIndex(ArrayList<String> invertedData,
        int[][] docno) {
        int i, j;
        for (i = 0; i < invertedData.size(); i++) {
            System.out.print(invertedData.get(i) + "\t");
            for (j = 1; j <= docno[i][0]; j++)
                System.out.print(docno[i][j] + "\t");
            System.out.print("\n");
        }
    }

    public static void indexing(String fname, ArrayList<String>
invertedData, int[][] docno, int fileno) {
        BufferedReader br;
        try {
            br = new BufferedReader(new FileReader(fname));
            String data = "", line = br.readLine();
            while (line != null) {
                data += line + " ";
                line = br.readLine();
            }
            String[] st = data.split("[ ,.]");
            String currenttoken = null;
            int i = 0;
            while (i < st.length) {
                currenttoken = st[i];
                int indx = invertedData.indexOf(currenttoken);
                if (indx == -1) {
                    invertedData.add(currenttoken);
                    indx = invertedData.indexOf(currenttoken);
                    docno[indx][0] = 1;
                    docno[indx][1] = fileno;
                }
                i++;
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

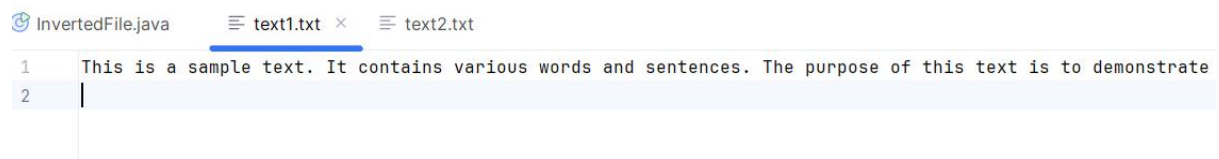
```

        } else {
            docno[indx][docno[indx][0] + 1] = fileno;
            docno[indx][0] += 1;
        }
        i += 1;
    }
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Output:

Text1.txt:



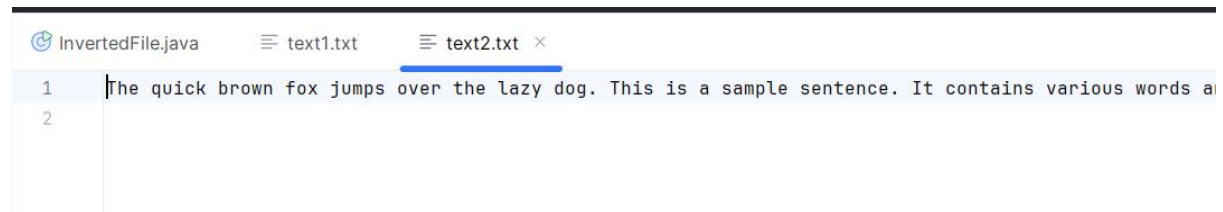
The screenshot shows an IDE window with three tabs: InvertedFile.java, text1.txt, and text2.txt. The text1.txt tab is active and displays the following text:

```

1 This is a sample text. It contains various words and sentences. The purpose of this text is to demonstrate
2 |

```

Text2.txt:



The screenshot shows an IDE window with three tabs: InvertedFile.java, text1.txt, and text2.txt. The text2.txt tab is active and displays the following text:

```

1 The quick brown fox jumps over the lazy dog. This is a sample sentence. It contains various words a
2 |

```

```

"C:\Program Files\Java\jdk-22\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA
Community Edition 2024.1.4\lib\idea_rt.jar=58044:C:\Program Files\JetBrains\IntelliJ IDEA
Community Edition 2024.1.4\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -
Dsun.stderr.encoding=UTF-8 -classpath
F:\BE\LP3\GroupA_Assignment_3_InvertedFiles\out\production\GroupA_Assignment_3_InvertedFil
es InvertedFile

```

ENTER TOTAL NO OF FILES:

2

ENTER FILE 1 NAME:

text1.txt

ENTER FILE 2 NAME:

text2.txt

This	1	2							
is	1	1	2						
a	1	2							
sample	1	2							
text	1	1	1	1					
	1	1	1	1	1	1	1	2	2
It	1	1	2						
contains		1	2						
various	1	2							
words	1	1	2						
and	1	1	1	2					
sentences		1							
The	1	1	2						
purpose		1							
of	1	1	1						
this	1								
to	1	1							
demonstrate		1							
the	1	1	1	1	1	1	2		
functionality		1							
conflation		1							
algorithm		1	1						
will	1	1							
remove	1	1							
stop	1								
suffixes	1								
count	1								
frequency		1							
each	1								
word	1								
provide	1								

insights 1
into 1
analysis 1
process 1
Feel 1
free 1
modify 1
observe 1
results 1
quick 2
brown 2
fox 2
jumps 2
over 2
lazy 2
dog 2
sentence 2
punctuation 2
marks 2

Process finished with exit code 0